🌾 Annadata — Food Donation & Distribution System

Complete Project Documentation & Presentation Guide

PART 1 — PROJECT OVERVIEW

What is Annadata?

Annadata (meaning "*the one who provides food*" in Sanskrit) is a desktop application built to connect food donors with recipients and manage daily food distribution across community service locations.

The system solves a real-world problem: - Food from restaurants, events, and households goes to waste - NGOs and shelters struggle to find timely food supply - No organized digital system exists to track food donations at a local level

Annadata bridges this gap by providing a role-based platform where: - **Donors** post available food - **Recipients** (NGOs, shelters, individuals) browse and claim it - **Admins** monitor the entire operation and manage schedules

---

Core Features

Feature	Description
Role-Based Login	3 roles: Admin, Donor, Recipient — each with a dedicated dashboard
Food Donations	Post food with category, quantity, expiry, source type
Claim Requests	Recipients can request/claim available donations
Emergency Aid	Recipients post urgent food needs for disasters or crises
Distribution Schedule	Weekly grid (Mon–Sun × Morning/Afternoon/Evening) for community kitchens, temples, shelters, etc.
Source Analytics	Visual charts showing donations by source (Individual / Organization / Government)
Notifications	Real-time in-app notifications for actions between roles
User Management	Admin can activate/deactivate accounts

PART 2 — TECHNOLOGY STACK

⚠️ Important Clarification

This project does **NOT** use Spring Boot. It is a **standalone desktop application** built with:

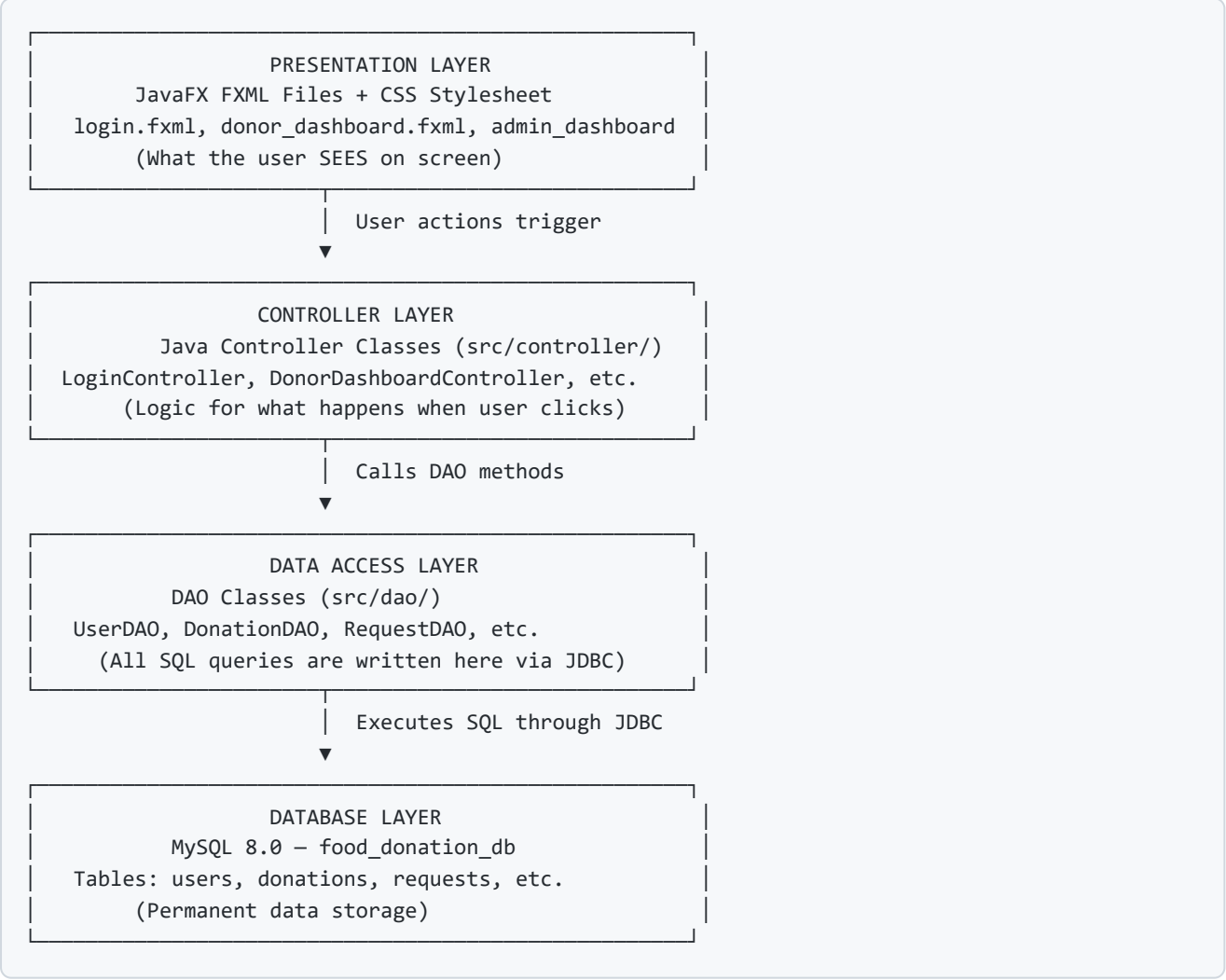
Technology	Role in Project
Java 21	Core programming language for all logic
JavaFX 21	UI framework for the desktop interface
FXML	XML-based layout files for each screen
CSS (JavaFX)	Styling the UI components (green theme)
JDBC	Java Database Connectivity — connects Java directly to MySQL
MySQL 8.0	Relational database storing all application data
BCrypt (jBCrypt)	Password hashing library for secure login
MySQL Connector/J	JDBC driver JAR that enables Java to talk to MySQL

Why not Spring Boot?

Spring Boot is a **web/server framework** — it runs in a browser. This project is a **desktop app** that runs as an installed program on a PC, which is why JavaFX is used instead.

PART 3 — ARCHITECTURE

Layered Architecture (MVC Pattern)



Model Layer (src/model/)

Between the DAO layer and Controller layer sits the **Model layer** — plain Java classes that represent data: - `User.java`, `Donation.java`, `Request.java`, `EmergencyAid.java` - `DistributionPlace.java`, `DistributionSchedule.java`, `Notification.java`

These are called **POJOs** (Plain Old Java Objects) — they just hold data with getters and setters.

PART 4 — HOW JAVA CONNECTS TO MYSQL

This is the most technically important part of the project.

Step 1 — The JDBC Driver

The file `lib/mysql-connector-j-8.4.0.jar` is the **bridge** between Java and MySQL.

Without this JAR, Java has no idea how to speak MySQL's language. When the app starts, Java loads this driver:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

This line tells Java: "*Load the MySQL driver so we can open connections.*"

Step 2 — DatabaseConnection.java

This file creates the actual connection to MySQL using a **Singleton pattern** (only one connection exists at a time):

```
connection = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/food_donation_db?allowPublicKeyRetrieval=true&useSSL=false&server
    "root",   // username
    "root"    // password
);
```

Breaking down the connection URL: | Part | Meaning | |---|---| | `jdbc:mysql://` | Protocol — tells Java to use the MySQL JDBC driver | | `localhost` | The MySQL server is on the same computer | | `3306` | MySQL's default port number | | `food_donation_db` | The database to connect to | | `allowPublicKeyRetrieval=true` | Required for MySQL 8.0 security handshake | | `useSSL=false` | No encrypted tunnel needed (local connection) | | `serverTimezone=UTC` | Avoid timezone mismatch errors |

Step 3 — DatabaseInitializer.java

On first startup, this class reads `sql/schema.sql` and executes it line by line to create all tables automatically. This means team members never need to manually create tables.

Step 4 — DAO Layer (Data Access Objects)

Every DAO file is responsible for one table. It uses `PreparedStatement` (not raw SQL strings) to prevent SQL injection:

```
String sql = "SELECT * FROM users WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, email);   // The ? is safely replaced here
ResultSet rs = pstmt.executeQuery();
```

The `?` placeholder ensures that user input is never directly embedded in SQL — preventing attacks.

Step 5 — ResultSet to Model Object

After running a query, MySQL returns a `ResultSet`. The DAO converts each row into a Java model object:

```
User user = new User();
user.setId(rs.getInt("id"));
user.setName(rs.getString("name"));
user.setEmail(rs.getString("email"));
// ... and so on
return user;
```

The controller then receives this `User` object and uses it to populate the UI.

PART 5 — HOW JavaFX WORKS

FXML + Controller Binding

JavaFX uses FXML (similar to HTML) to define what the screen looks like, and Java Controllers to handle what happens.

Example — Login Screen:

login.fxml (Layout):

```
<TextField fx:id="emailField" promptText="Email"/>
<PasswordField fx:id="passwordField" promptText="Password"/>
<Button text="Login" onAction="#handleLogin"/>
```

LoginController.java (Logic):

```
@FXML private TextField emailField;
@FXML private PasswordField passwordField;

@FXML
public void handleLogin() {
    String email = emailField.getText().trim();
    String password = passwordField.getText();
    User user = new UserDAO().login(email, password);
    if (user != null) {
        SessionManager.setCurrentUser(user);
        // Navigate to role dashboard
    }
}
```

The `@FXML` annotation tells JavaFX: *"Link this Java variable to the matching `fx:id` in the FXML file."*

Scene Switching

When a user logs in, the scene changes:

```
FXMLLoader loader = new FXMLLoader(getClass().getResource("/fxml/donor_dashboard.fxml"));
Stage stage = (Stage) emailField.getScene().getWindow();
stage.setScene(new Scene(loader.load()));
```

This replaces the current screen content with the new screen — like navigating between pages.

SessionManager.java

Since JavaFX doesn't have sessions like web apps do, we built a simple static class to hold the currently logged-in user in memory:

```
public class SessionManager {
    private static User currentUser;

    public static void setCurrentUser(User user) { currentUser = user; }
    public static User getCurrentUser() { return currentUser; }
    public static void logout() { currentUser = null; }
}
```

Every screen can call `SessionManager.getCurrentUser()` to know who is logged in.

PART 6 — DATABASE DESIGN

Database: food_donation_db

Table 1: users

Stores all accounts (admin, donor, recipient).

Column	Type	Description
id	INT (PK, Auto)	Unique user ID
name	VARCHAR(100)	Full name
email	VARCHAR(100)	Login email (unique)
password_hash	VARCHAR(255)	BCrypt hashed password
role	ENUM	'admin', 'donor', 'recipient'
phone	VARCHAR(15)	Contact number
address	TEXT	Physical address
is_active	TINYINT(1)	1=active, 0=deactivated
created_at	TIMESTAMP	Registration time

Table 2: donations

Every food item posted by a donor.

Column	Type	Description
id	INT (PK, Auto)	Unique donation ID
donor_id	INT (FK → users)	Who donated it
title	VARCHAR(200)	Name of the donation
description	TEXT	Details
category	ENUM	cooked/raw/packaged/beverages/dog_food/other
quantity	INT	How much food
unit	VARCHAR(50)	kg / plates / litres / packets
expiry_date	DATETIME	When it expires
pickup_address	TEXT	Where to collect
source_type	ENUM	individual / organization / government
source_name	VARCHAR(200)	Name of the source
status	ENUM	available / accepted / completed / expired / cancelled
created_at	TIMESTAMP	When it was posted

Table 3: requests

When a recipient claims a donation.

Column	Type	Description
id	INT (PK, Auto)	Request ID
donation_id	INT (FK → donations)	Which donation
recipient_id	INT (FK → users)	Who is claiming
status	ENUM	pending / accepted / rejected / completed
message	TEXT	Note from recipient
created_at	TIMESTAMP	When claimed

Table 4: emergency_aids

Urgent food requests posted by recipients.

Column	Type	Description
id	INT (PK, Auto)	Emergency ID
recipient_id	INT (FK → users)	Who posted
title	VARCHAR(200)	Emergency title
emergency_type	ENUM	disaster_relief / medical_emergency / community_crisis / natural_calamity
urgency	ENUM	critical / high / medium
people_affected	INT	Number of people
location	TEXT	Where the emergency is
required_items	TEXT	What food is needed
status	ENUM	open / responding / partially_fulfilled / fulfilled / closed

Table 5: distribution_places

Fixed community locations that receive daily food.

Column	Type	Description
id	INT (PK, Auto)	Place ID
name	VARCHAR(200)	Place name
category	ENUM	dog_food_service / old_age_home / temple / orphanage / community_kitchen / hospital / school
address	TEXT	Location
contact_person	VARCHAR(100)	Manager name
capacity	INT	How many people it serves
is_active	TINYINT(1)	Currently active?

Table 6: distribution_schedule

The weekly schedule linking places to time slots.

Column	Type	Description
id	INT (PK, Auto)	Schedule ID
place_id	INT (FK → distribution_places)	Which place
day_of_week	VARCHAR(10)	Monday / Tuesday etc.
time_slot	ENUM	morning / afternoon / evening
food_type	VARCHAR(100)	Type of food delivered
quantity_target	INT	Target quantity
assigned_volunteer	VARCHAR(100)	Volunteer name
donation_id	INT (FK → donations, nullable)	Linked donation if any
status	ENUM	scheduled / assigned / in_transit / delivered / completed
recurring	TINYINT(1)	Does it repeat every week?

Table 7: notifications

In-app notifications for all users.

Column	Type	Description
id	INT (PK, Auto)	Notification ID
user_id	INT (FK → users)	Who receives it
message	TEXT	Notification text
type	VARCHAR(50)	donation / request / emergency / system
read_status	TINYINT(1)	0=unread, 1=read
related_id	INT	ID of the related record
created_at	TIMESTAMP	When sent

PART 7 — SECURITY

Password Hashing with BCrypt

Passwords are **never stored as plain text**. Before saving to the database, the password goes through BCrypt hashing:

```
User types: "admin123"
BCrypt converts to: "$2a$10$N9qo8uLOickgx2ZMRZoMye..."
MySQL stores: "$2a$10$N9qo8uLOickgx2ZMRZoMye..."
```

When logging in:

```
User types: "admin123"
BCrypt compares it against the stored hash → returns true/false
Java never decrypts the hash — it only compares
```

This means even if someone steals the database, they cannot recover any passwords.

PreparedStatements (SQL Injection Prevention)

Instead of building SQL strings like:

```
// DANGEROUS — never do this:
String sql = "SELECT * FROM users WHERE email = '" + email + "'";
```

We always use:

```
// SAFE — parameterized query:
String sql = "SELECT * FROM users WHERE email = ?";
pstmt.setString(1, email);
```

PART 8 — PROJECT FOLDER STRUCTURE

```
food-donation-system/
│
├── src/                          ← All Java source code
│   ├── app/
│   │   └── Main.java             ← Entry point, starts JavaFX app
│   ├── db/
│   │   ├── DatabaseConnection.java   ← Creates MySQL connection (Singleton)
│   │   └── DatabaseInitializer.java  ← Runs schema.sql on first launch
│   ├── model/                    ← Data classes (POJOs)
│   │   ├── User.java
│   │   ├── Donation.java
│   │   ├── Request.java
│   │   ├── EmergencyAid.java
│   │   ├── DistributionPlace.java
│   │   ├── DistributionSchedule.java
│   │   └── Notification.java
│   ├── dao/                      ← SQL query classes
│   │   ├── UserDAO.java
│   │   ├── DonationDAO.java
│   │   ├── RequestDAO.java
│   │   ├── EmergencyAidDAO.java
│   │   ├── DistributionPlaceDAO.java
│   │   ├── DistributionScheduleDAO.java
│   │   └── NotificationDAO.java
│   ├── controller/               ← Screen logic
│   │   ├── LoginController.java
│   │   ├── RegisterController.java
│   │   ├── DonorDashboardController.java
│   │   ├── RecipientDashboardController.java
│   │   ├── AdminDashboardController.java
│   │   ├── CreateDonationController.java
│   │   └── ...more controllers
│   └── util/
│       ├── PasswordUtil.java     ← BCrypt hashing
│       ├── SessionManager.java   ← Holds logged-in user
│       └── AlertUtil.java        ← Popup dialogs
│
├── resources/
│   ├── fxml/                     ← Screen layouts (like HTML for desktop)
│   │   ├── login.fxml
│   │   ├── donor_dashboard.fxml
│   │   └── ...
│   └── css/
│       └── styles.css            ← Green theme styling
│
├── sql/
│   ├── schema.sql                ← Creates all 7 tables
│   └── dumps/                    ← Live data exports per table
│
├── lib/                          ← External JAR libraries
│   ├── javafx-controls-21.0.2.jar
│   ├── mysql-connector-j-8.4.0.jar
│   ├── jbcrypt-0.4.jar
```

```
│   └── ...
│
├── out/                          ← Compiled .class files (auto-generated)
├── compile.bat                   ← One-click compile
└── run.bat                       ← One-click run
```

PART 9 — HOW THE TECHNOLOGIES CONNECT

Complete Request Flow (Example: Donor posts a donation)

```
1. DONOR clicks "New Donation" in the sidebar
        ↓
2. DonorDashboardController.showCreateDonation()
   → Loads create_donation.fxml into the content area
        ↓
3. Donor fills form: title, category, quantity, expiry, address
        ↓
4. Donor clicks "Submit"
        ↓
5. CreateDonationController.handleSubmit()
   → Reads all TextField/ComboBox values
   → Creates a new Donation object:
        Donation d = new Donation();
        d.setTitle("Fresh Bread");
        d.setCategory("packaged");
        d.setDonorId(SessionManager.getCurrentUser().getId());
        ↓
6. DonationDAO.create(d)
   → Opens MySQL connection via DatabaseConnection.getConnection()
   → Runs PreparedStatement:
        INSERT INTO donations (donor_id, title, ...) VALUES (?, ?, ...)
   → MySQL writes the row to the donations table
        ↓
7. Success alert shown to donor
   Donor sees new donation in "My Donations" list
```

Complete Login Flow

```
1. User types email + password → clicks Login
        ↓
2. LoginController.handleLogin()
        ↓
3. UserDAO.login(email, password)
   → SELECT * FROM users WHERE email = ?
   → MySQL returns a row
   → PasswordUtil.checkPassword(typed, storedHash) → true/false
        ↓
4. If true:
   → SessionManager.setCurrentUser(user)
   → Load dashboard based on user.getRole():
        "admin"     → admin_dashboard.fxml
        "donor"     → donor_dashboard.fxml
        "recipient" → recipient_dashboard.fxml
        ↓
5. Dashboard opens with user's name in header
```

PART 10 — PRESENTATION GUIDE

Suggested Presentation Flow (10–15 minutes)

Slide 1 — Problem Statement

"Every day, tonnes of food goes to waste while millions go hungry. There is no organized system to connect food donors with those who need it in real-time."

Slide 2 — Solution: Annadata

- Desktop application (runs offline)
- Three roles: Admin, Donor, Recipient
- Manages donations, claims, emergency requests, and weekly distribution

Slide 3 — Tech Stack

Layer	Technology
Programming Language	Java 21
UI Framework	JavaFX 21 + FXML
Database	MySQL 8.0
DB Connection	JDBC + MySQL Connector/J
Password Security	BCrypt (jBCrypt)
Styling	JavaFX CSS

Slide 4 — Architecture Diagram

Show the 4-layer diagram from Part 3 of this document.

Slide 5 — Database Design

Show the 7 tables and their relationships (FK connections).

Slide 6 — Live Demo

Walk through these flows: 1. Login as Admin → show overview stats 2. Login as Donor → post a new food donation 3. Login as Recipient → browse and claim a donation 4. Admin → Source Charts tab 5. Admin → Weekly Distribution Schedule

Slide 7 — Security Features

- BCrypt password hashing
- PreparedStatements (no SQL injection)
- Role-based access control (each role sees only their screens)
- Session management

Slide 8 — Challenges Faced

- Connecting JavaFX to MySQL without a framework
- Building dynamic UI (tables, cards, charts) entirely in Java code
- BCrypt hash verification flow
- Managing session state in a desktop app without HTTP sessions

Slide 9 — Future Scope

- Mobile app version (Android/iOS)
- SMS/email notification integration

- Google Maps integration for pickup locations
- Cloud MySQL (accessible over internet so no local DB needed)
- Multi-language support (Hindi, Tamil, etc.)

PART 11 — KEY INTERVIEW / VIVA QUESTIONS

Q: What is JDBC? A: Java Database Connectivity — a standard Java API that allows Java programs to connect and execute SQL queries on relational databases like MySQL, PostgreSQL, etc.

Q: What is the difference between Statement and PreparedStatement? A: Statement embeds values directly into SQL (unsafe, SQL injection risk). PreparedStatement uses `?` placeholders and separates data from the query (safe, precompiled).

Q: What is BCrypt? A: A password hashing algorithm. Unlike MD5/SHA, BCrypt is deliberately slow and includes a "salt" (random data) to make brute-force attacks impractical.

Q: What is MVC? A: Model-View-Controller. Model = data (Java model classes), View = FXML layouts, Controller = Java controller classes. It separates concerns so each part is independent.

Q: What is a Singleton pattern? A: A design pattern that ensures only one instance of a class exists. Used in `DatabaseConnection.java` so only one MySQL connection is open at a time.

Q: What is a DAO? A: Data Access Object. A design pattern where all database operations for one table are grouped into one class. Keeps SQL out of the UI code.

Q: What is JavaFX? A: A Java framework for building desktop GUIs. Uses FXML for layouts (similar to XML/HTML) and CSS for styling, with Java controllers for logic.

Q: Why not use Spring Boot? A: Spring Boot is a web application framework — it serves pages in a browser. Annadata is a desktop app meant to run locally without internet, making JavaFX the appropriate choice.

Document prepared for: Annadata — Food Donation & Distribution System Technology: Java 21 + JavaFX 21 + MySQL 8.0 + JDBC

🌟 SPECIAL MODULE DEEP-DIVE: DONATIONS & CLAIM REQUESTS

Assigned Module for Aashray — Architecture, Code Walkthrough & Presentation Defense

---

1. Overview of the Module

In the Annadata system, the **Donations and Claim Requests** module forms the core operational heart of the application. Without this module, food cannot be listed, discovered, requested, or transferred.

It fulfills the two fundamental user journeys: 1. **Donor Journey**: A donor lists surplus food with expiry time, quantity, category, and pickup details, monitors its status, and approves or rejects claim requests. 2. **Recipient Journey**: A verified recipient (shelter, community kitchen, or individual) browses available donations in real-time, submits claim requests with custom notes, and tracks claim approval.

---

2. The 8 Core Classes in Detail

Class 1: `Donation.java` (Model Layer)

- **Role**: Plain Old Java Object (POJO) representing a row in the `donations` table.
- **Key Attributes**:
- `id`: Unique identifier (auto-increment in MySQL).
- `donorId`: Foreign key to `users(id)`.
- `title`, `description`: Details of the food.
- `category`: Category string (`cooked`, `raw`, `packaged`, `beverages`, `dog_food`, `other`).
- `quantity`, `unit`: Measurement of food (e.g., `50 plates`, `20 kg`).
- `expiryDate` (`LocalDateTime`): Crucial for food safety.
- `pickupAddress`: Physical pickup location.
- `sourceType` (`individual`, `organization`, `government`), `sourceName`.
- `status`: Lifecycle state (`available`, `accepted`, `completed`, `expired`, `cancelled`).
- `donorName`: Additional field populated during SQL `JOIN` with the `users` table for display.
- **Why it matters**: Encapsulates donation data so controllers and DAOs pass typed objects rather than loose strings or arrays.

Class 2: `Request.java` (Model Layer)

- **Role**: POJO representing a claim request for a donation in the `requests` table.
- **Key Attributes**:
- `id`: Primary key.
- `donationId`: Foreign key to `donations(id)`.
- `recipientId`: Foreign key to `users(id)`.
- `status`: State of request (`pending`, `accepted`, `rejected`, `completed`).
- `message`: Note from recipient explaining their requirement or logistics.
- `createdAt`: Timestamp when request was submitted.
- `donationTitle`, `donorName`, `recipientName`: Joined display fields for dashboards.
- **Why it matters**: Tracks the negotiation between donor and recipient before physical food transfer.

Class 3: `DonationDAO.java` (Data Access Layer)

- **Role**: Handles all database operations related to the `donations` table using JDBC and `PreparedStatement`.
- **Key Methods**:
- `create(Donation d)`:
  - Executes `INSERT INTO donations (...) VALUES (?, ?, ...)`.
  - Sets parameters using `pstmt.setString()`, `pstmt.setInt()`, and converts `LocalDateTime` to `java.sql.Timestamp`.
- `findAvailable()`:
  - Executes: `sql SELECT d.*, u.name as donor_name FROM donations d LEFT JOIN users u ON d.donor_id = u.id WHERE d.status = 'available' AND d.expiry_date > NOW() ORDER BY d.created_at DESC`
  - Filters out expired food and already-claimed donations automatically at the database level!
- `findByDonor(int donorId)`:
  - Returns all donations created by a specific donor.
- `findById(int id)`:
  - Fetches complete details of a single donation joined with donor's name.
- `updateStatus(int id, String status)`:
  - Transitions donation state (e.g., from `available` to `accepted` or `completed`).
- `countBySourceType()` & `countMonthlyBySource()`:
  - Aggregates donation stats for the Admin analytics dashboard.
- **Why it matters**: Isolates all SQL queries so that if database schema or driver changes, the UI code remains untouched.

Class 4: `RequestDAO.java` (Data Access Layer)

- **Role**: Manages all SQL persistence and retrieval for claim requests.
- **Key Methods**:
- `create(Request r)`:
  - Executes `INSERT INTO requests (donation_id, recipient_id, status, message, created_at) VALUES (?, ?, ?, ?, ?)`.
  - Defaults status to `'pending'`.
- `findByDonation(int donationId)`:
  - Executes `SELECT r.*, u.name as recipient_name FROM requests r JOIN users u ON r.recipient_id = u.id WHERE r.donation_id = ?`.
  - Allows the donor to see who requested their food and read their messages.
- `findByRecipient(int recipientId)`:
  - Executes a multi-table JOIN across `requests`, `donations`, and `users` so the recipient sees donation titles and donor names in their "My Claims" screen.
- `updateStatus(int id, String status)`:
  - Changes request status to `'accepted'` or `'rejected'`.
- **Why it matters**: Protects against SQL injection using parameterized queries while maintaining referential integrity.

Class 5: `CreateDonationController.java` (Controller Layer)

- **Role**: Backing controller for `create_donation.fxml`.
- **How it works**:
- `initialize()`: Populates dropdown ComboBoxes for `categoryCombo` (`cooked`, `raw`, `packaged`, etc.) and `sourceTypeCombo`. Auto-populates donor address and name from `SessionManager.getCurrentUser()`.
- `handleSubmit()`:
  1. Extracts values from `titleField`, `categoryCombo`, `quantityField`, `expiryPicker`, etc.
  2. Validates that no required fields are blank and validates that quantity is numeric (`Integer.parseInt`).
  3. Constructs `Donation` object with current donor's user ID from `SessionManager`.
  4. Calls `donationDAO.create(d)`.
  5. Displays user-friendly confirmation using `AlertUtil.showInformation()` and resets form fields.
- `handleCancel()`: Clears the input fields.

Class 6: `DonationDetailsController.java` (Controller Layer)

- **Role**: Adaptive details view for both Donors and Recipients (`donation_details.fxml`).

- **Dynamic Dual-Role Logic**:
- Receives the donation object and current role via `setDonation(Donation d, String role)`.
- **When viewed by a Recipient**:
  - Shows donation specs (category, quantity, expiry, donor source).
  - Enables the `recipientBox` with a message input field and a "Claim This Donation" button.
  - If status is not `'available'`, disables the claim button.
  - When claimed: calls `requestDAO.create()`, disables button, and creates an in-app notification for the donor via `NotificationDAO`.
- **When viewed by a Donor**:
  - Enables the `donorBox` and calls `loadRequestsForDonor()`.
  - Populates all incoming claim requests with `Accept` and `Reject` action buttons.
  - Accepting updates both `requests.status = 'accepted'` and `donations.status = 'accepted'`.
  - Provides options to "Mark Completed" or "Cancel Donation".

Class 7: `DonorDashboardController.java` (Controller Layer)

- **Role**: Coordinates the main interface for logged-in donors (`donor_dashboard.fxml`).
- **Key Responsibilities**:
- Displays welcome message with donor's name.
- Shows real-time unread notification count badge via `notificationDAO.countUnread()`.
- Generates stat cards: Total Donations, Active Donations, Completed, Expiring Soon.
- Renders recent donations dynamically as styled cards with action buttons (View Details, Edit, Cancel).
- Navigation handlers:
  - `showCreateDonation()`: Loads `create_donation.fxml` into central `contentArea`.
  - `showMyDonations()`: Queries `donationDAO.findByDonor(userId)` and builds donation cards.
  - `showTodaySchedule()`, `showEmergencies()`, `showNotifications()`.
  - `handleLogout()`: Clears `SessionManager` and redirects to login screen.

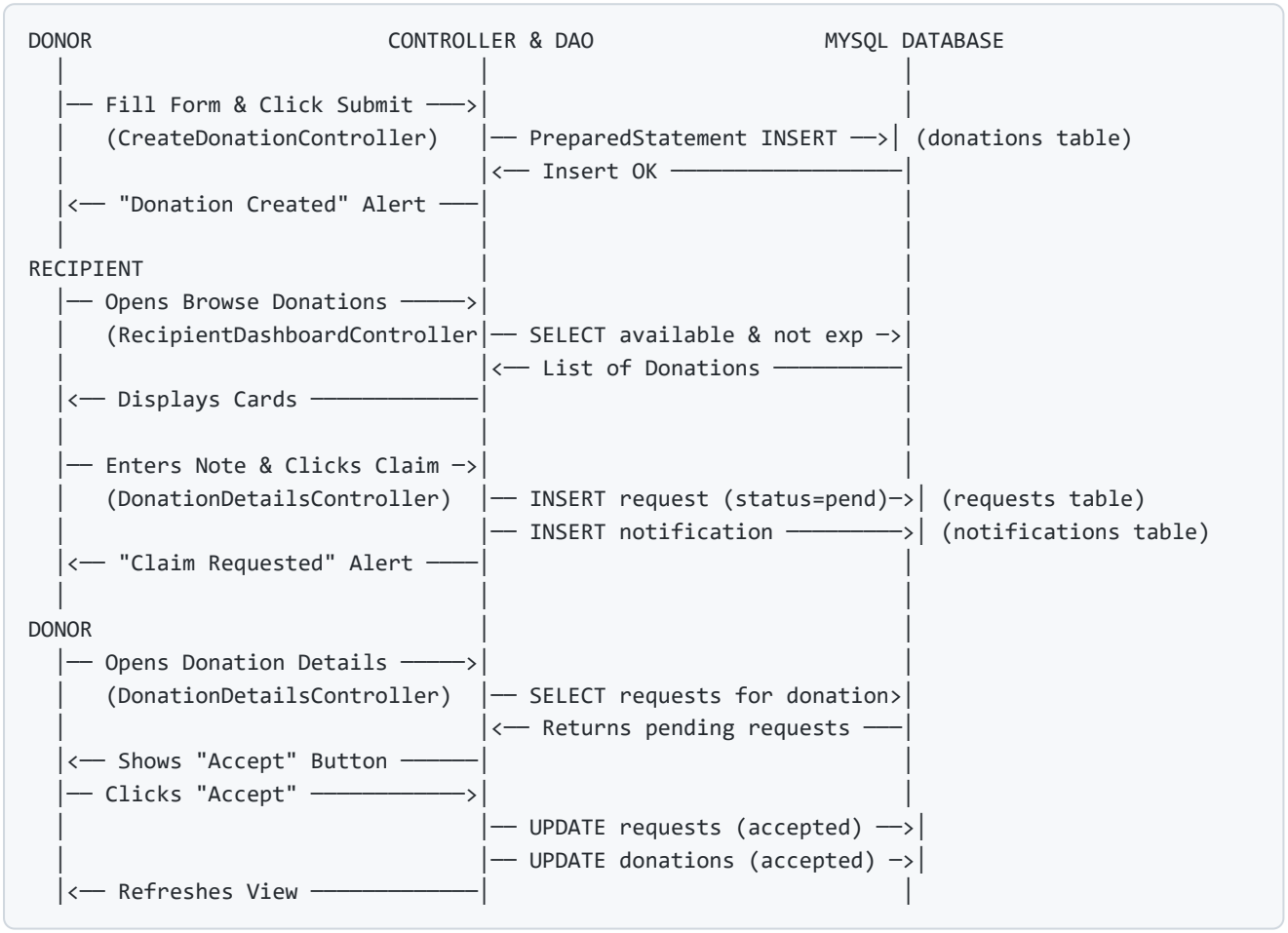
Class 8: `RecipientDashboardController.java` (Controller Layer)

- **Role**: Coordinates the primary interface for recipients/NGOs (`recipient_dashboard.fxml`).
- **Key Responsibilities**:
- `showBrowse()`:
  - Calls `donationDAO.findAvailable()` to fetch all non-expired, unreserved donations.
  - Provides real-time keyword search and category filtering.
  - Renders responsive cards displaying food category badge, quantity, expiry time, and pickup address.
  - Clicking "View Details" opens `DonationDetailsController` with recipient capabilities.

- `showMyClaims()`:
  - Calls `requestDAO.findByRecipient(userId)` to display all claims made by this recipient.
  - Displays claim status badges (`PENDING`, `ACCEPTED`, `REJECTED`).
- Access to Emergency Aid requests and weekly distribution schedule.

---

3. End-to-End Sequence Diagram



---

4. What to Say in Your Presentation & Viva

1. Elevator Pitch of Your Module:

> *"My module is 'Donations and Claim Requests' — it is the primary business engine of Annadata. It governs how food items are created, validated, published, searched, claimed by recipients, and approved by donors. I implemented 4 controllers, 2 DAOs, and 2 Model classes managing the complete lifecycle of a food donation."*

2. If Asked: "How do you ensure food safety and prevent expired food from being claimed?"

> *"In `DonationDAO.findAvailable()`, we run a condition `WHERE status = 'available' AND expiry_date > NOW()`. This ensures expired donations are automatically filtered out at the database query level, preventing recipients from requesting outdated food."*

3. If Asked: "How do you handle role-based behavior in the details view?"

> *"Instead of creating two duplicate screens for donors and recipients, `DonationDetailsController` takes a `viewerRole` parameter in `setDonation()`. If the user is a recipient, it shows the claim form and message box. If the user is a donor, it loads all pending requests with 'Accept' and 'Reject' action buttons."*

4. If Asked: "Why use PreparedStatements in DonationDAO and RequestDAO?"

> *"Every query uses `PreparedStatement` with parameterized placeholders `?`. This protects the application against SQL injection attacks and improves performance because the database engine pre-compiles the SQL execution plan."*